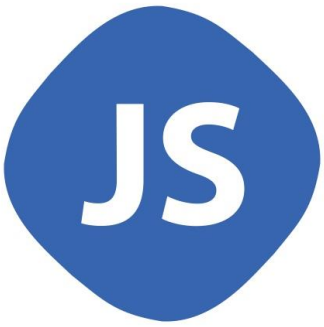


Типы данных

- Динамическая типизация
- Типы данных
- Примитивные типы данных
- Ссылочные типы данных

Динамическая типизация

В процессе работы компьютерные программы манипулируют различными значениями, каждое из которых может быть обработано в языке программирования и относится к определённому **типу данных**.

В JavaScript типы данных можно разделить на две категории:

- **простые** (примитивные) типы;
- **составные** (ссылочные) типы.

JavaScript – это **слабо типизированный** или динамический язык программирования, который позволяет определять типы данных, осуществлять синтаксический анализ и компиляцию "на лету", на этапе выполнения программы. Это значит, что вам не нужно определять тип переменной заранее. Тип определится автоматически во время выполнения программы.

Таким образом, в различных участках программы одна и та же переменная может принимать значения разных типов.

example_1. Слабая типизация JavaScript

```
<script>
	// инициализируем переменную значениями разных типов
	var value = "string"; // строка
	value = 7; // число
```



```
    value = {name: "Иван", login: "master"}; // объект
    value = ["php", "javaScript", "python"]; // массив
    value = true; // булев
    value = null; // нулевой
</script>
```

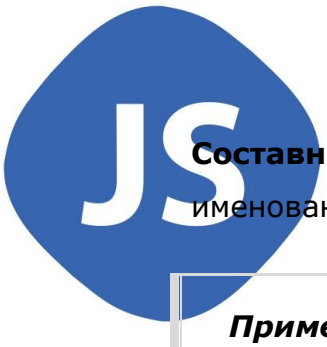
К сведению. Массивы и объекты большие темы для изучения. Но в рамках этого урока коснемся **основ**, необходимых для понимания изучаемого материала. Самостоятельными заданиями закрепим вопросы, обязательные к усвоению на этом этапе.

Типы данных

Стандарт ECMAScript определяет следующие типы данных:

Простые (примитивные) типы:

- **числовой** (number) – числовой тип данных в формате 64-битного числа двойной точности с плавающей запятой;
- **строковый** (string) – представляет собой последовательность символов, используемых для представления текста;
- **логический** (boolean) — может принимать два возможных значения, иногда называемых истиной (true) и ложью (false);
- **нулевой** (null) – значение null представляет ссылку, которая указывает, обычно намеренно, на несуществующий или некорректный объект или адрес;
- **неопределённый** (undefined) – обозначает предопределенную глобальную переменную, инициализированную неопределенным значением;
- **символ** (symbol) — тип данных, экземпляры которого уникальны и неизменяемы (новый в ECMAScript 6).



Составной (ссылочный) тип – объект (object). Объект - это коллекция именованных значений, которые обычно называют свойствами объекта.

> ***Примечание***. К составным типам относят объект и его производные (массивы, функции и некоторые другие).

Прежде чем рассмотреть каждый тип данных, познакомимся сначала с оператором typeof.

Оператор typeof

С помощью оператора **typeof** можно получить тип переменной. JavaScript определяет спецификацию оператора следующим образом:

```
typeof[()]expression[()];
```

а это значит, что оба способа написания оператора являются правильными:

```
let str = "Long Live Rock 'N' Roll";
let num = 3.14;
console.log(typeof(str));  // string
console.log(typeof num);  // number
```

Оператор **typeof** возвращает строку, описывающую тип данных переменной.

example_2. Оператор typeof

```
<script>
    // экспериментируем с типами данных
    var value = "string"; // строка
    console.log(typeof(value));  // string
    value = 7;  // простое число
    console.log(typeof(value));  // number
    value = {name: "Иван", login: "master"};  // объект
```

```
    console.log(typeof(value));  // object
    value = ["php", "javaScript", "python"];  // массив
    console.log(typeof(value));  // object (обратить внимание)
    value = true; // булев тип
    console.log(typeof(value));  // boolean
    value = null; // специальный тип null
    console.log(typeof(value));  // object
</script>
```

К сведению.

- Для значения **массива** оператор typeof возвращает значение **object**. Позже мы рассмотрим, что массив является частным случаем объекта.
- Для значения **null** оператор typeof возвращает значение **object** несмотря на то, что согласно спецификации JavaScript значение null представляет отдельный тип.

Разница между примитивными и составными типами

Разница между примитивными и составными типами данных проявляется при **копировании их значений**.

Вспомним. Переменная — это поименованная область памяти компьютера, отводимая под хранение некоторого значения.

Когда переменной присваивается значение простого типа, то в переменную записывается само значение (например число).

Когда мы присваиваем **переменную простого типа** в другую переменную, она **копирует значение**. В результате каждая переменная будет иметь **своё значение** и изменения в одной из переменных **никак не сказывается** на значении другой переменной.



Поэкспериментируем с переменной простого типа.

example_3. Присваивание переменных простого типа

```
<script>
	var num1 = 5; // создаем переменную простого типа (число)
	var num2 = num1; // присваиваем переменную num1 переменной num2,
	// при этом значение 5 скопировалось в переменную (num2)
	document.write("<h2>Начальные значения переменных</h2>");
	// выводим значения переменных
	document.write("num1 - " + num1); // 5
	document.write("<br>num2 - " + num2); // 5
	document.write("<h2>Изменяем значение переменной num1</h2>");
	num1 = 100; // изменяем значение
	// выводим значения переменных
	document.write("num1 - " + num1); // 100
	document.write("<br>num2 - " + num2); // 5
</script>
```

Важно. Когда переменной присваивается значение простого типа, то в переменную записывается **само значение**.

Когда переменной **присваиваем значение составного** (ссылочного) типа, то в переменную записывается **ссылка на значение** (ссылка на объект).

Когда мы присваиваем одну переменную (в значении которой ссылка на составное значение) другой переменной, то происходит **копирование ссылки** на составное значение. В результате **обе переменные ссылаются на одно и то же составное значение** и изменения в значении одной из переменных **будут сказываться** на другой переменной.

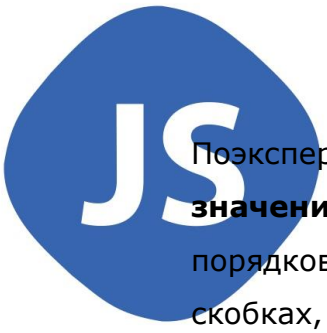
Важно. Когда переменной присваиваем значение составного типа, то в переменную записывается **ссылка на значение** (ссылка на объект).

Поэкспериментируем с переменной типа **объект**. Объект состоит из набора свойств в виде пар **ключ: значение**. Для обращения к свойствам объекта используется **точечная нотация**:

```
// инициализируем объект
var obj1 = {x:10};
// выводим свойство объекта в браузер
document.write( obj.x ); // 10
```

example_4. Присваивание переменных составного типа (объект)

```
<script>
	// создадим объект
	var obj1 = {x:10}; // объект - переменная ссылочного типа
	var obj2 = obj1; // присваиваем одну переменную другой,
	// при этом значение не копируется,
	// обе переменные будут ссылаться на одну область памяти
	document.write("<h2>Начальные значения свойств объектов</h2>");
	document.write("obj1.x - " + obj1.x); // 10
	document.write("<br>obj2.x - " + obj2.x); // 10
	document.write("<h2>Изменяем значение свойства объекта
	obj1</h2>");
	// изменим свойство объекта obj1
	// свойство объекта obj2 должно измениться автоматически
	obj1.x = 25;
	document.write("obj1.x - " + obj1.x); // 25
	document.write("<br>obj2.x - " + obj2.x); // 25
</script>
```



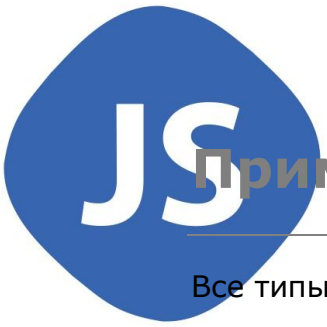
Поэкспериментируем с переменной типа **массив**. Массив состоит из набора **значений**. Для обращения к значениям используется индекс, обозначающий порядковый номер значения в массиве. Индекс указывается в квадратных скобках, отсчет индексов начинается с **0**:

```
// инициализируем массив
var arr = ["php", "javaScript", "python"]
// выводим второй элемент ммассива
document.write(arr[1]); // javaScript
```

example_5. Присваивание переменных составного типа (массив)

```
<script>
	// создадим массив
	// массив - переменная ссылочного типа
	var arr1 = ["php", "javaScript", "python"]; //исходный массив
	var arr2 = arr1; // присваиваем массив
	// при этом значения исходного массива не копируется,
	// обе переменные будут ссылаться на одну область памяти
	document.write("<h2>Начальные значения второго элемента массивов</h2>");
	document.write("arr1[1] - " + arr1[1]); // javaScript
	document.write("<br>arr2[1] - " + arr2[1]); // javaScript
	document.write("<h2>Изменяем значение второго элемента первого массива</h2>");
	// изменим второй элемент первого массива
	// второй массив должен измениться соответственно
	arr1[1] = "JS";
	document.write("<br>arr1[1] - " + arr1[1]); // JS
	document.write("<br>arr2[1] - " + arr2[1]); // JS
</script>
```





Примитивные типы данных

Все типы данных в JavaScript, кроме объектов, являются неизменяемыми или иммутабельными (англ. immutable). Это значит, что их значения не могут быть модифицированы, а только перезаписаны новым другим значением. Например, строки нельзя корректировать посимвольно – их можно переписать только полностью. Значения таких типов называются "примитивными".

Простейшие данные, с которыми может оперировать программа, называются **литералами**. Литералы — это числа или строки, которые используются для представления значений в JavaScript. Предоставляемая информация может быть самой разнообразной, а поэтому значения бывают различных типов. Простейшие типы данных в JavaScript называются основными типами данных: числа, строки и логические значения. Все они относятся к "примитивным".

Тип данных Number

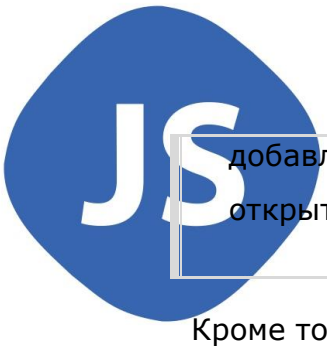
В JavaScript, нет различия между целым числом и числом с плавающей точкой – по сути, JavaScript представляет все числа в качестве значения с плавающей точкой:

- **Целые** числа. Можно использовать как положительные, так и отрицательные числа. Диапазон используемых чисел: от -2^{53} до 2^{53}.
- **Дробные** числа (числа с плавающей точкой). Можно использовать как положительные, так и отрицательные числа. Для чисел с плавающей точкой используется тот же диапазон: -2^{53} до 2^{53}.

```
var age = 37;  // целое
var price = 215.7;  // дробное
```

Число, находящееся непосредственно в коде программы, называется **числовым литералом**.

> ***Примечание***. В последних стандартах EcmaScript для представления **очень больших** целых чисел, которые выходят за пределы диапазона типа number



добавлен тип **BigInt**. При желании информацию о типе можно посмотреть в открытых источниках.

Кроме того, JavaScript содержит специальные числовые значения:

- **NaN** (не число или ошибка вычислений). Является результатом некорректной математической операции над недопустимыми данными, такими как строки или неопределенное значение.
- **Infinity** (положительная бесконечность). Используется, если положительное число слишком велико и не может быть представлено в JavaScript.
- -**Infinity** (отрицательная бесконечность). Используется, если отрицательное число слишком велико и не может быть представлено в JavaScript.
- **±0** (положительный и отрицательный 0). JavaScript различает положительный и отрицательный ноль.

Помимо десятичных целых литералов JavaScript распознает шестнадцатеричные значения.

Тип данных String

Тип **String** представляет строки. Для определения строк применяются кавычки, JavaScript позволяет использовать **двойные** ("), **одинарные** ('), **косые** (`) кавычки.

Единственно ограничение: тип закрывающей кавычки должен быть тот же, что и тип открывающей, то есть кавычки должны быть парными.

```
let user = "Евгений&Татьяна";
let vk = 'pechora_PROger';
let language = `JavaScript`;
```

Если внутри строки встречаются кавычки, то их необходимо **экранировать слешем** (\).



Например, пусть у нас есть текст "Бюро "Рога и копыта"". Вывод текста в браузер может быть выполнен несколькими способами:

```
// экранируем дополнительные кавычки
let company = "Бюро \"Рога и копыта\"";
// можно использовать другой вид кавычек
let company1 = "Бюро 'Рога и копыта'";
// или так
let company2 = 'Бюро "Рога и копыта"';
```

Инттерполяция

Использование косых кавычек позволяет применять такой прием как **интерполяция** - **встраивание данных в строку**.

Например:

```
let user = "Eugene";
// интерполируем переменную в строку
let msg = `Hello our dear ${user}!`;
console.log(msg);  // Hello our dear Eugene!
```

Для встраивания значений выражений (значений переменных и констант) в строку перед выражением ставится знак доллара $, после которого в фигурных скобках указывается выражение. Так, в примере выше **${user}** означает, что в этом месте строки надо встроить значение переменной **user**.

Подобным образом можно встраивать и больше количество данных:

example_6. Пример интерполяции переменных

```
<script>
      let login = "master";
      let id = 3157;
      let access = true;
```

```
    let msg = `Login: ${login} (id=${id}), allow full access:
    ${access}`;
    // вывод в консоль сформированного сообщения
    console.log(msg);
    // Login: master (id=3157), allow full access: true
</script>
```

Тип данных Boolean

Тип **Boolean** представляет булевы или логические значения **true** (верно) и **false** (ложно):

```
let isEmail = true;
let course = false;
```

Тип данных Null

Нулевой тип (null) содержит единственное специальное значение – **null**.

Ключевое слово **null** невозможно использовать в качестве имени функции или переменной. Значение null является ссылкой на "пустой" объект и имеет специальное назначение – обычно оно используется для инициализации переменной, которой впоследствии будет **присвоено** значение.

Оператор typeof для значения null возвращает строку **object**, что свидетельствует о том, что значение null является специальным "пустым" объектом.

example_7. Специальный тип данных null

```
<script>
    // объявление переменной
    let login = null;
    console.log(login);
```

```
    // ...
    // инициализация переменной
    login = "master";
    console.log("Значение - ", login);
    console.log("Тип данных - ", typeof(login));
</script>
```

Тип данных Undefined

Неопределенный тип (undefined) образует свой собственный тип, который содержит единственное специальное значение – **undefined**. Такое значение имеет переменная, объявленная с помощью операторов **var** или **let**, но не инициализированная:

```
let login = null; // объявление и инициализация переменной
let email; // объявление переменной
console.log("login: ", login);  // login: null
console.log("email: ", email);  // email: undefined
```

Значение **undefined** возвращается при обращении к **переменной**, которой никогда не **присваивалось значение**, а также к **несуществующему свойству объекта или элементу массива**.

Следует отметить, что переменная со значением undefined отличается от переменной, которая вообще не определена:

example_8. Тип данных undefined

```
<script>
    let login = null; // объявление и инициализация переменной
    let email; // объявление переменной
    // let phone; // переменная не объявлена
    console.log("login: ", login);  // login: null
```

```
    console.log("email: ", email); // email: undefined
    console.log("phone: ", phone) // Uncaught ReferenceError: phone
    is not defined
</script>
```

Следующий пример может показаться нелогичным, но факт остается фактом, оператор **typeof** и для **неинициализированной**, и для **необъявленной** переменной возвращает значение **undefined**:

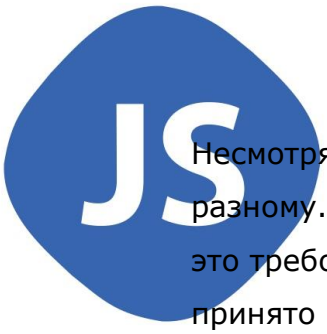
example_9. Особенности применения typeof

```
<script>
    var value;  // переменная объявляется, но имеет значение
    undefined
    // var more; // эта переменная не объявляется
    alert(typeof(value));  // выведет undefined
    alert(typeof(more));  // тоже выведет undefined
</script>
```

В приведенном примере переменная **value** объявлена, но в неё ничего не записано, поэтому её значение как раз и есть **undefined**. Переменная **more** не объявлена – её, по сути, нет. Тем не менее, **typeof** возвращает строку **undefined** в обоих случаях.

Определенный смысл в этом, конечно, есть, с любой из этих переменных невозможно выполнить никаких операций, хотя технически они совершенно разные.

> ***Примечание***. Рекомендуется всегда выполнять **инициализацию объявленной переменной**. В таком случае вы будете знать, что оператор **typeof** возвращает **undefined** из-за того, что переменная не была объявлена, а не потому, что она не инициализирована.



Несмотря на то, что значения **null** и **undefined** связаны, используются они по-разному. **Не следует** явно присваивать переменной значение **undefined**. К null это требование не относится, когда необходимый объект недоступен, ему принято присваивать значение **null**.

Тип данных Symbol

Символ (symbol) является нововведением JavaScript начиная с ECMAScript версии 6. Символ – это уникальное, **неизменяемое**, **примитивное** значение, которое служит для создания уникальных идентификаторов.

> ***Примечание***. Тип данных символ используют чаще всего при работе с объектами. В нашем курсе этот тип данных нам не понадобится (по крайней мере пока планов таких нет).

Ссылочные типы данных

Тип данных Object

В дополнение к рассмотренным выше примитивным типам данных JavaScript поддерживает составной тип данных – **объект** (Object).

Объект состоит из многих значений (либо элементарных, таких как числа и строки, либо сложных, например массивов или других объектов), которые называются свойствами объекта. Доступ к свойствам осуществляется по имени свойства (иногда говорят "по ключу").

Имя свойства является строкой, поэтому можно считать, что объекты связывают строки со значениями. Вместе эти фрагменты информации образуют пары "ключ-значение".

В JavaScript для создания объекта можно использовать синтаксис:

- объектного **литерала**:

```
let book = {};
```

- **конструктора Object**():

```
let book = new Object();
```

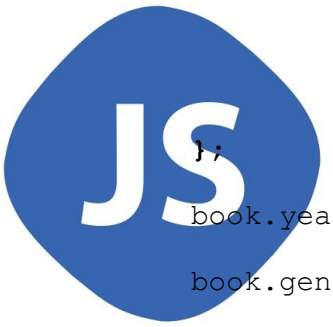
Создание объекта с помощью **объектного литерала** начинается с определения обычной переменной. В правой части этой инструкции записывается литерал объекта – заключенный в фигурные скобки **{ }** список разделенных запятой пар **имя: значение**. Имя свойства и значение свойства отделены друг от друга двоеточием:

example_10. Литеральное создание объекта

```
<script>
	// литеральное объявление объекта
	let book = {
		"name": "Мастер и Маргарита",
		"author": "М.Булгаков",
		"year": 1940,
		"genre": "Мистика",
		"bestseller": true
	};
	// выводим в консоль
	console.log(typeof book);// тип переменной book
	console.log(book); //описание объекта
</script>
```

Можно сначала создать объект, затем добавить к нему недостающие свойства по мере необходимости:

```
let book = {
	"name": "Мастер и Маргарита",
	"author": "М.Булгаков",
```



```
};
book.year = 1940;
book.genre = "Мистика";
book.bestseller = true;
```

Кавычки в именах свойств объектов необходимы если:

- имя свойства **зарезервировано**;
- имя свойства содержит **специальные символы** или пробелы.

Пример:

```
var myObject = {
      "e-mail": "1@mail.ru",  // запрещенный символ `-`
      "class": "person", // зарезервированное слово `class``
      valid: "happy" // кавычки не требуются
};
```

Второй способ создания объектов связан с использованием **конструктора Object**. При этом сначала используется выражение **new Object**(), а затем определяются и инициализируются свойства полученного объекта:

example_11. Создание объекта конструктором

```
<script>
      // объявление объекта конструктором
      let book = new Object();
      // инициализация свойств объекта
      book.name = "Мастер и Маргарита";
      book.author = "М.Булгаков";
      book.year = 1940;
      book.genre = "Мистика";
      book.bestseller = true;
      // выводим в консоль
```



```
    console.log(typeof book);  // тип переменной book
    console.log(book);  //описание объекта
</script>
```

Обратиться к свойству объекта можно посредством **точечной нотации**. Точка между именем объекта и свойством указывает на принадлежность свойства данному объекту. Свойства объекта работают как переменные – в них можно сохранять значения и по мере необходимости считывать их.

example_12. Дозапись и обращение к свойствам объекта

```
<script>
    // литеральное объявление объекта
    let book = {
        "name": "Мастер и Маргарита",
        "author": "М.Булгаков"
    };
    // добавление свойств объекта
    book.year = 1940;
    book.genre = "Мистика";
    book.bestseller = true;
    // выводим в браузер
    document.write("<h2>", book.name, "</h2>");
    document.write("<p>Автор: ", book.author, "</p>");
    document.write("<p>Год: ", book.year, "</p>");
    document.write("<p>Жанр: ", book.genre, "</p>");
    document.write("<p>Бестселлер: ", book.bestseller, "</p>");
</script>
```

Тип данных Array

Объекты позволяют хранить данные со строковыми ключами. Но довольно часто веб-разработчику необходима упорядоченная коллекция данных, в



которой присутствуют 1-й, 2-й, 3-й элементы и т.д. Например, она понадобится нам для хранения списка чего-либо: товаров, пользователей, услуг и т.д.

В этом случае использовать объект неудобно, так как он не предоставляет методов управления порядком элементов. Для хранения упорядоченных коллекций существует особая структура данных, которая называется **массив** (**Array**).

Существует два варианта синтаксиса создания пустого массив. Массив можно создать используя:

- **квадратные** скобки:

```
let arr = [];
```

- **конструктор Array**():

```
let arr = new Array();
```

Практически всегда используется первый вариант синтаксиса. В скобках мы можем указать начальные значения элементов:

```
let lang = ["JavaScript", "php", "perl"];
```

В массиве могут храниться элементы **любого типа**. Элементы массива нумеруются, начиная с нуля.

Получить элемент, можно указав его номер в квадратных скобках:

```
alert(lang[0]); // JavaScript
alert(lang[1]); // php
alert(lang[2]); // perl
```

Мы можем заменить элемент:

```
lang[0] = "python";
console.log(lang); //  ['python', 'php', 'perl']
```

…или добавить новый элемент к существующему массиву:

```
lang[3] = "python";
```



```
console.log(lang); //  ['javaScript', 'php', 'perl', 'python']
```

example_13. Работа с массивами

```
<script>
	// инициализация массива
	let book = ["Мастер и Маргарита", "М.Булгаков", 1940];
	book[3] = "Мистика";
	book[4] = true;
	// вывод значений массива в браузер
	document.write("<h2>", book[0], "</h2>");
	document.write("<p>Автор:book[1], "</p>");
	document.write("<p>Год:book[2], "</p>");
	document.write("<p>Жанр:book[3], "</p>");
	document.write("<p>Бестселлер:book[4], "</p>");
</script>
```

На практике чаще всего случается, что один тип отлично расширяет другой.

Например, жанр фильма "Мастер и Маргарита" не определяется только как "Мистика". И если вспомнить, что значением свойства объекта может быть любой тип, можно свойству **genre** присвоить массив, определяющий набор жанровых характеристик фильма.

example_14. Массив как значение свойства объекта

```
<script>
	// литеральное объявление объекта
	let book = {
		"name": "Мастер и Маргарита",
		"author": "М.Булгаков",
		"year": 1940,
		"genre": ["Мистика", "Триллер", "Драма", "Детектив"],
```

```
        "bestseller": true
    };
    // формируем строку для вывода в браузер,
    // используем синтаксис косых кавычек
    let line = `
        <h2> ${book.name} </h2>
        <p><i>Автор:</i>  ${book.author} <p>
        <p><i>Год: </i> ${book.year} <p>
        <p><i>Жанр: </i>
        <ul>
            <ol>${book.genre[0]}</ol>
            <ol>${book.genre[1]}</ol>
            <ol>${book.genre[2]}</ol>
            <ol>${book.genre[3]}</ol>
        </ul>
        <p>
        <p><i>Бестселлер: </i>${book.bestseller} `;
    // выводим строку в браузер
    document.write (line);
</script>
```

В веб-разработке структуры обрабатываемых данных могут быть намного сложнее. Но первый шаг (и совсем не маленький) к изучению мы сделали. Закрепим материал выполнением самостоятельных работ.

Самостоятельные работы:

- Объявление простых переменных.
- Объявление ссылочных типов (массива, объекта).
- Интерполяция переменных в строки с косыми кавычками.
- Массивы как значения свойств объектов.